UNIVERSITY OF RWANDA
COLLEGE OF SCIENCE AND TECHNOLOGY
SCHOOL OF ICT
DEPARTMENT OF COMPUTER AND SOFTWARE ENGINEERING
ACADEMIC YEAR: 2025 - 2026
YEAR OF STUDY: III
DATE: On Saturday 21st March 2026
MODULE: MOBILE APPLICATION SYSTEMS AND DESIGN
REG. NUMBER: 223001019

LAB 5: LOCAL STORAGE IN FLUTTER
POSTS MANAGER WITH SQLite

Q1. Which dependencies did you use and why?

* sqflite: ^2.3.2:
It provides a full SQLite implementation for flutter on Android, iOS and Desktop. It was chosen because:
  - The app must work 100% offline, no server, no internet.
  - Supports full SQL: Insert, Select, Update, Delete, where, order by, like
  - Data persists permanently in a ".db" file on the device across app restarts.
  - All queries run on a background thread and return Future<T>, so the UI never freezes.

* path: ^1.9.0:
A utility for building file paths correctly across different operating systems. SQLite needs to know where to store ".db" file, and path separators differ between Android, iOS, and Desktop. Used like this:

```
final path = join(await getDatabasesPath(), 'posts_manager.db');
```

Without it you'd have to handle OS specific separators manually and risk placing the file in the wrong location.

* intl: ^0.19.0:
Handles date and time formatting in the UI. Flutter's raw "DateTime.toString()" produces something like 2026-03-20 09:15:42.000, which is not user-friendly. With "intl":
"DateFormat('MMM d, YYY. h:mm a').format(post.CreatedAt)"
=> "Mar 20, 2026 · 9:15 AM"

* cupertino_icons: ^1.0.6:
Provides Apple-style (iOS) icons via the CupertinoIcons class. This is actually include by default in every new flutter project generated by "flutter create". Even thoughthe app primarily uses MaterialDesign(Icons.*), this package ships with the default template and has negligible over head, so it is kept in.

Q Why SQlite is necessary for local (offline) storage?
SQlite is as serverless, self-contained relational database engine that stores all data in a single file on the device. Unlike shared preferences (key-value only) or plain files (no structured queries), SQlite enables:

- Run full SQL queries - select, insert, update, delete, where, order by
- Store relational data with typed columns (integer, text, real, blob, null).
- Persist data permanently across app restarts without internet.
- Handle multiple records efficiently with indexed lookups.
- Enforce data integrity via NOT NULL constraints and conflict algorithms.

Q2. How do you handle database ~~connection~~? exceptions?

A custom DatabaseException class wraps all errors so the UI layer receives one consistent exception type regardless of the underlying cause:

```
class DatabaseException implements Exception {
    final String message;
    final dynamic originalError;
    DatabaseException(this.message, [this.originalError]);
    @override
    String toString() => 'DatabaseException: $message';
}
```

* Handle Database not initialized.
The "DatabaseHelper" uses a singleton with a lazy getter. The database connection is only opened the first time it is needed, and the getter guards against calling the database before it is ready:

```
static Database? _database;
Future<Database> get database async {
    if (_database != null && _database!.isOpen) return _database!;
    _database = await _initDatabase();
    return _database!;
}
```

```
Future < Database > _initDatabase() async {
  try {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, 'posts_manager');
    return await openDatabase(path, version: 1, onCreate: _createTable)
  } catch(e){
      throw DatabaseException('Failed to initialize database', e);
    }
  }
```

If 'openDatabase()' fails for any reason, disk full, corrupted file, permissions issue. the "catch" block wraps the raw platform error into a clean "DatabaseException('Failed to initialise database')". Every subsequent CRUD method calls "await database" first, so they all benefit from this guard automatically.

* Handle Insert/update/delete Errors

Each write operation is individually wrapped in its own try-catch. the pattern is the same across all three:

```
// INSERT
Future<int> insertPost(Post post) async {
  try {
    final db = await database;
    final map = post.toMap()..remove('id');
    return await db.insert(
      'posts',
      map,
      conflictAlgorithm: ConflictAlgorithm.abort,
      );
  } on DatabaseException {
      rethrow;
  } catch(e) {
      throw DatabaseException('Failed to insert post', e);
    }
  }

// UPDATE
Future<int> updatePost(Post post) async {
  try {
      final db = await database;
      final rows = await db.update('posts', post.toMap(),
          where: 'id=?', whereArgs: [post.id]);
      if(rows==0) throw DatabaseException('Post with id ${post.id}
                                          no found');
      return rows;
  } on DatabaseException {
      rethrow;
  } catch(e) { throw DatabaseException('Failed to update post', e); }}
```

```
// DELETE
Future<int> deletePost(int id) async{
  try{
    final db = await database;
    final rows = await db.delete('posts', where: 'id=?', whereArgs: [id]);
    if(rows==0) throw DatabaseException('post with id $id not found');
    return rows;

  } on DatabaseException{
    rethrow;
  } catch(e) {
   throw DatabaseException('Failed to delete post', e);
   }
}
```

Two things worth noting here. First, "on DatabaseException {rethrow;}" is listed before the general "catch(e)", this prevents a "DatabaseException" thrown intentionally (like the "not found" check) from being rewrapped into a second "DatabaseException". Second, checking 'rows==0' after update and delete catches the silent failure case where SQlite would appear to succeed even though nothing actually changed.
In the UI, every call is also wrapped so the user always gets feedback:

```
try{
  await _db.deletePost(post.id!);
  // show success snackBar
} catch (e) {
    scaffoldMessenger.of(context).showSnackBar(
    SnackBar(context: Text('Error: $e'), backgroundColor: colors.red),
    );

  }
```

* Handle Invalid or Corrupted Data

This is handled at two separate layers, the UI and the database layer, as a defence-in-depth approach.

- Layer 1 – UI validation (FormField validators)

The form prevents the user from ever submitting bad data in the first place:

```
TextFormField(
      controller: _titleCtrl,
      validator: (v){
        if(v== null || v.trim().isEmpty) return 'Title is required';
        if(v.trim().length < 3) return 'Title must be atleast 3 characters';
        return null;
      },
    )
```

The form calls "_formKey.currentState!.validate()" before any database call is made, so invalid input never reaches the database layer at all.

• Layer 2 - DatabaseHelper guards

Even if somehow invalid data bypasses the UI (e.g., if the database is called programmatically), DatabaseHelper performs its own checks:

```
Future<int> insertPost(Post post) async {
  if (post.title.trim().isEmpty) throw DatabaseException('Title cannot be empty');
  if (post.body.trim().isEmpty) throw DatabaseException('Body cannot be empty');
  // Proceed with insert
}
```

• Corrupted data read from the database

When reading rows back out, "Post.fromMap()" uses safe parsing to prevent a single bad row from crashing the entire app:

```
factory Post.fromMap(Map<String, dynamic> map) {
    return Post(
        id: map['id'] as int?,
        title: map['title'] as String? ?? '',
        body: map['body'] as String? ?? '',
        author: map['author'] as String? ?? '',
        category: map['category'] as String? ?? '',
        // tryParse returns null on invalid; fallback to now()
        createdAt: DateTime.tryParse(map['created_at'] ?? '') ?? DateTime.now(),
        updatedAt: DateTime.tryParse(map['updated_at'] ?? '') ?? DateTime.now(),
    );
}
```

"DateTime.tryParse()" is used instead of "DateTime.parse()", the difference is that "parse()" throws an exception on a mal~~function~~formed string while "tryParse()" returns "null", which then falls back to "DateTime.now()". This means even if a timestamp in the database is corrupted or empty, the app degrades gracefully instead of crashing.

Q3. Explain sqlite in Flutter

* Database Vs Table.

Database and table are often confused but they operate at different levels.

• A database is the container, a single file stored on the device with a ".db" extension. In this app it lives at a path like:

→ "/data/user/0/com.example.offline_posts_manager/databases/posts_manager.db"

It is created once using "openDatabase()" and holds everything. Think of it like a single Excel workbook file sitting on your harddrive.

```
final path = join(await getDatabasesPath(), 'posts_manager.db');
final db = await openDatabase( path, version: 1, onCreate: _createTable,);
```

- A Table is a structured grid of rows and columns that lives inside the database, like a single sheet inside that workbook. Each column has a fixed name and data type. This app has one table called "posts", defined with SQL like this:

```
await db.execute('''
  CREATE TABLE posts (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    title       TEXT    NOT NULL,
    body        TEXT    NOT NULL,
    author      TEXT    NOT NULL,
    category    TEXT    NOT NULL DEFAULT 'General',
    created_at  TEXT    NOT NULL,
    updated_at  TEXT    NOT NULL
  )
'''
);
```

The relationship is simple, one database can contain many tables, but in this app only one is needed. You could later add "users" table or a "drafts" table inside the same database, "posts_manager.db" file without touching anything else.

Concept	Analogy	in this app
Database	The workbook file	posts_manager.db
Table	A sheet inside the workbook	posts
Row	A single line in the sheet	one post
Column	A labelled field in each row	title, author, etc.

* CRUD operations

CRUD stands for Create, Read, Update, Delete, - the 4 foundamental operations any data driven app needs. Each maps directly to a SQL statement and a sqflite method.

- Create - INSERT

  Adds a new row to a table. sqflite's db.insert() takes the table name and a Map<String, dynamic> where keys are columnnames.

```
Future<int> insertPost(Post post) async {
    final db = await database;
    final map = post.toMap()..remove('id');
    return await db.insert(
        'posts',
        map,
        conflictAlgorithm: ConflictAlgorithm.abort,
    );
}
```

The return value is the auto-generated 'id' of the new row. ConflictAlgorithm.abort means if something violates a constraint the insert fails loudly rather than silently.

- Read – SELECT

Retrieves rows from the table. db.query() is the high-level wrapper around SELECT:

```
//Read all posts, newest first
Future<List<Post>> getAllPosts() async {
 final db = await database;
 final maps = await db.query('posts', orderBy: 'created_at DESC');
 returns maps.map(Post.fromMap).toList();
}
//Search by keyword
Future<List<Post>> searchPosts(String keyword) async {
    final db = await database;
    final maps = await db.query(
      'posts',
      where: 'title LIKE? OR body LIKE? OR author LIKE?',
      whereArgs: ['%$keyword%', '%$keyword%', '%$keyword%'],
      );
    return maps.map(Post.fromMap).toList();
}
```

The "?" placeholders and "whereArgs" list are important, they prevent SQL injection by keeping user input separate from the query structure.

- Update – UPDATE

Modifies an existing row. db.update() targets specific row using where and whereArgs:

```
Future<int> updatePost(Post post) async {
    final db = await database;
    final rows = await db.update(
```

```
'posts',
post.toMap(),
where: 'id=?',
whereArgs: [post.id],
);
if(rows==0) throw DatabaseException('post not found');
return rows;
}
```

The return value is the number of rows affected. checking for "rows==0" catches the case where the id no longer exists in the database.

- Delete – DELETE

Removes a row permanently. db.delete() works the same way as update; target by where:

```
Future<int> deletePost(int id) async{
    final db = await database;
    final rows = await db.delete(
      'posts',
      where: 'id=?',
      whereArgs: [id],
    );
  if(rows==0) throw DatabaseException('Post not found');
  return rows;
}
```

* CRUD Summary:

operation	SQL equivalent	sqflite method	Returns
Create	INSERT INTO posts	db.insert()	New row 'id'
Read	SELECT * FROM posts	db.query()	List<Map>
Update	UPDATE posts SET ...	db.update()	Rows affected
Delete	DELETE FROM posts WHERE	db.delete()	Rows affected

* How Flutter interacts with the database asynchronously?

Flutter's entire UI runs on a single thread called the main isolate. this thread is responsible for drawing the screen, handling touch events, and running animations. if any operation on this thread takes too long, the UI freezes and the app becomes unresponsive.

CamScanner

SQlite queries involve reading and writing to disk which can be slow. If the database was called in a blocking way, the UI would freeze every time a query runs. This is a bad user experience.

The sqflite package solves this problem by running all queries on a separate background thread. It then communicates the result back to the main thread using Dart's Future System.

A Future is a value that does not exist yet but will be available at some point in the future. It is Dart's version of a promise. Instead of waiting and blocking, the app continues running and handles the result when it arrives.

The "async" keyword is written before a function to mark it as asynchronous. The "await" keyword is written before a Future to pause that function until the result arrives, without blocking the UI thread.

When "getAllPosts()" is called, the sqflite package sends the SQL query to a background thread. The UI thread remains free. The user can still scroll, tap and see animations. When the queries finishes, the result travels back to main thread as a completed Future.

After the data arrives, setState() is called. This tells flutter to rebuild the screen with the new data. This is how the list of posts appears on screen after loading.

Thread flow:

step1: setState is called and the screen shows a loading spinner.
step2: await db.query() is called and the SQL query is sent to the background thread.
step3: The background thread runs the query and reads data from the disk.
step4: The Future completes and the result is sent back to main thread.
step5: setState is called again and the screen rebuilds to show the list of posts.

Summary:

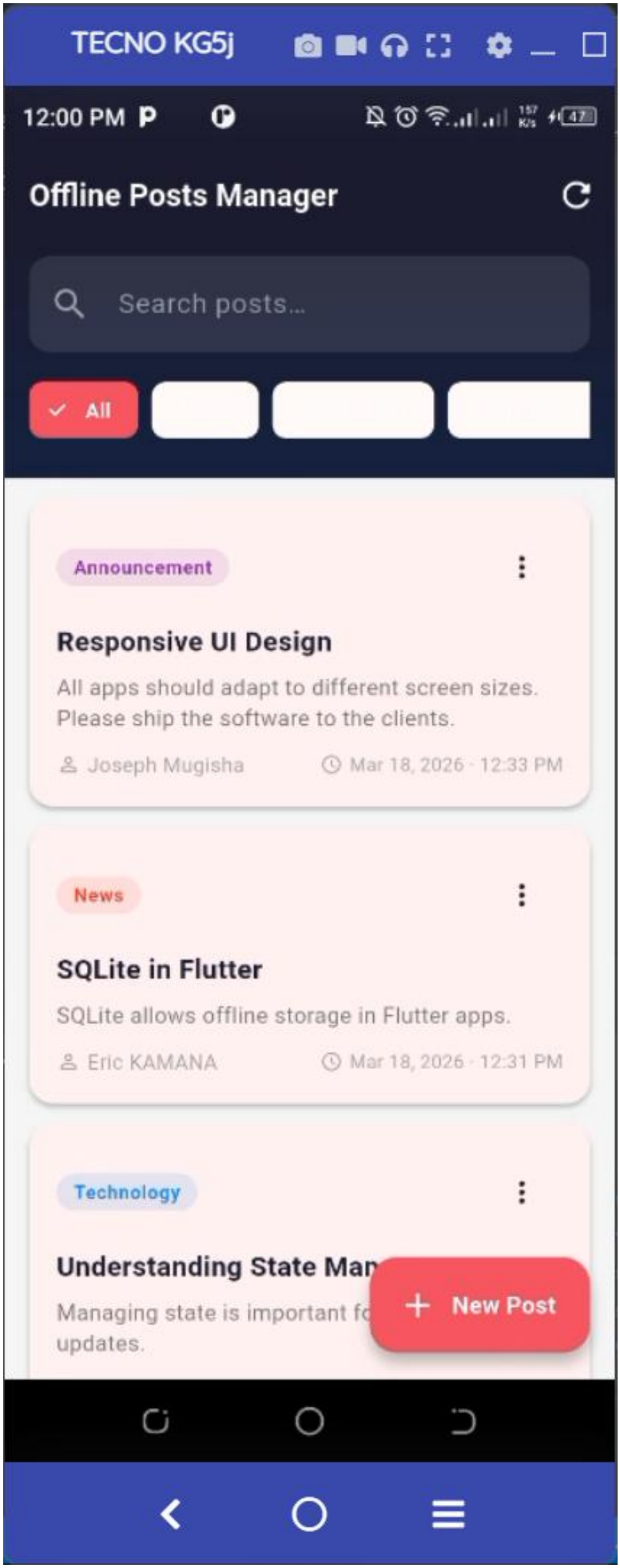
Every method in DatabaseHelper is written with async and every sqflite call uses await. This is not just a coding style. It is what keeps the app smooth and responsive. The UI thread is never blocked because all database work happens on a background thread and the result is delivered through a Flutter's Future when it is ready.

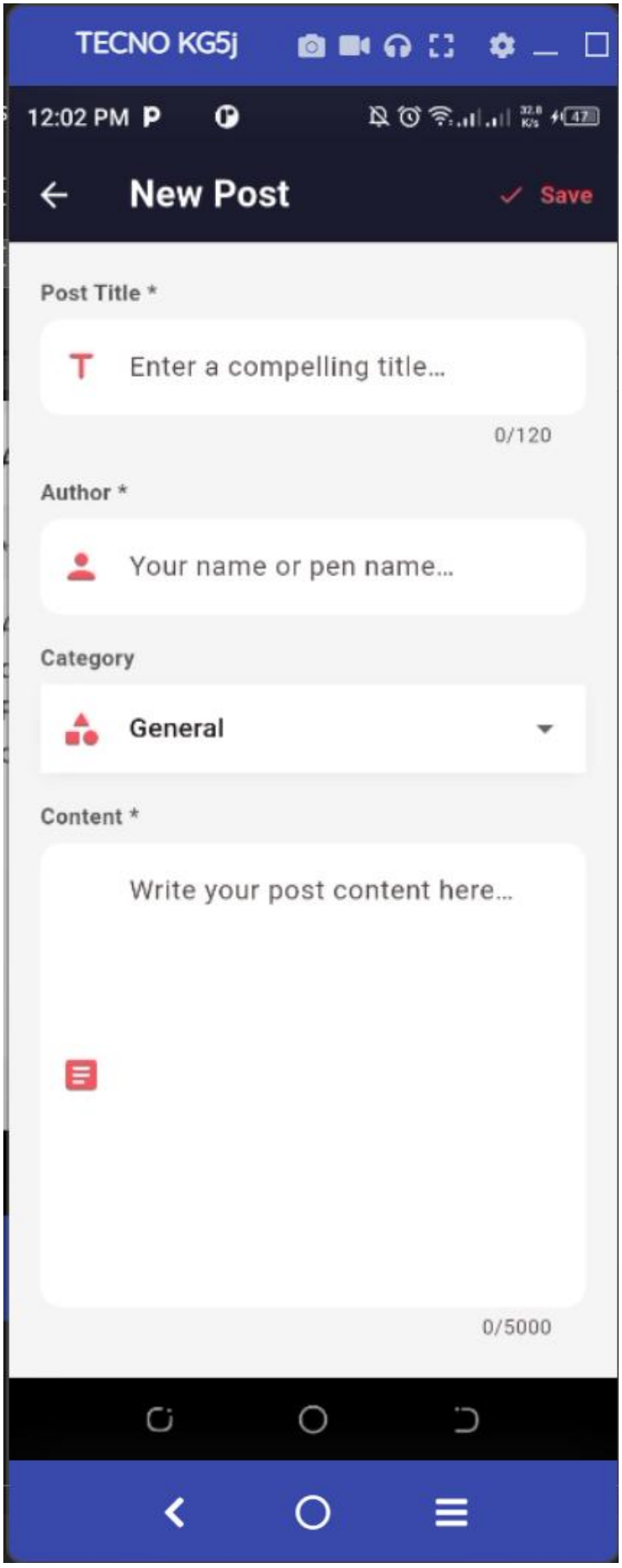
Flutter's async/await syntax lets developers write sequentially-looking code that is non-blocking:

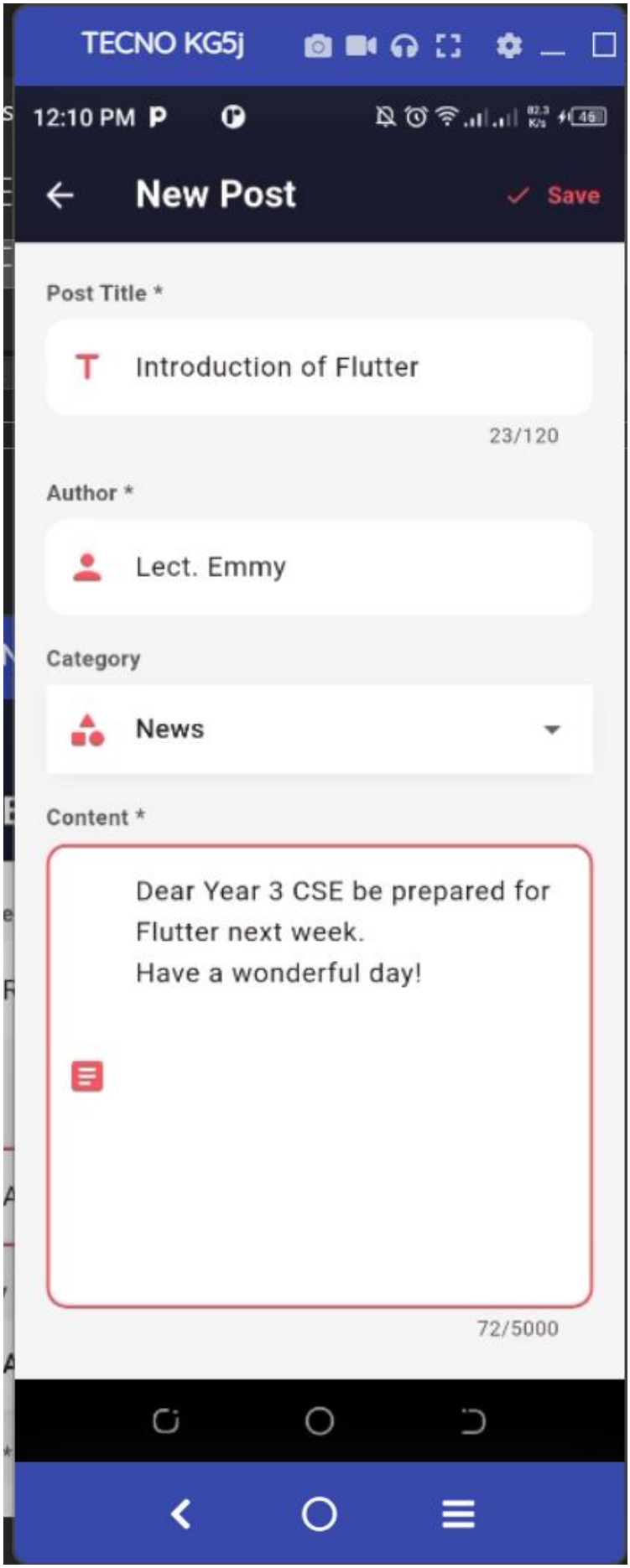
Example: Insert a post without blocking the UI

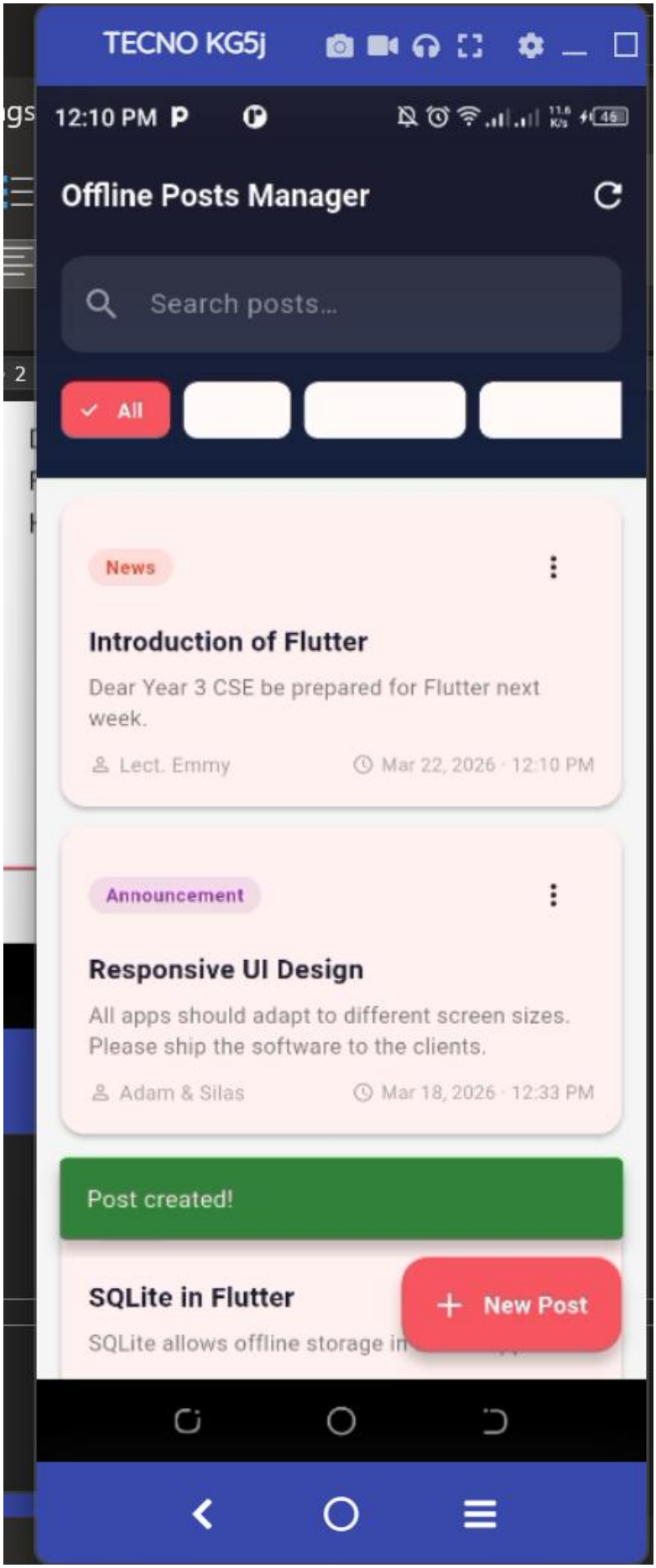
```
Future insertPost (Post post) async{
    final db = await database; // await the DB connection
    final id = await db.insert( // await the INSERT query
      'posts',
      post.toMap(),
      conflictAlgorithm: ConflictAlgorithm.abort,
    );
  return id;
}
```

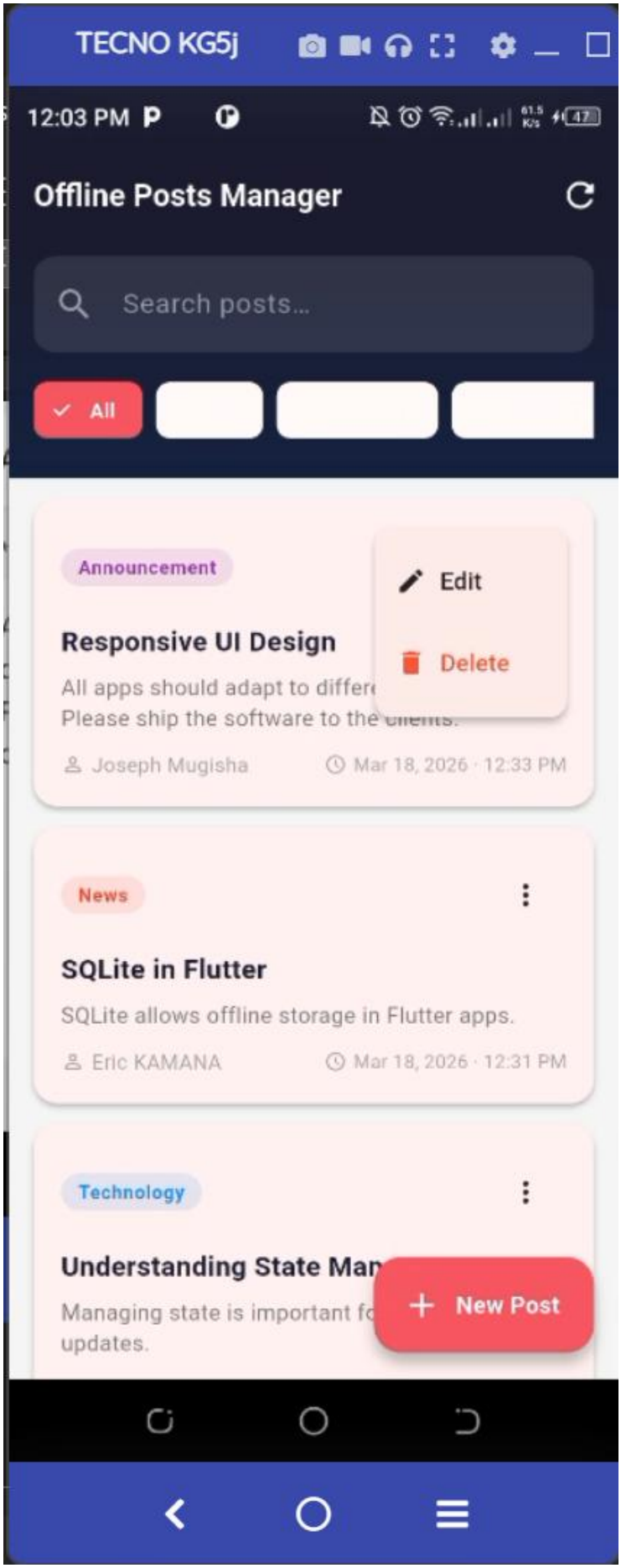
END

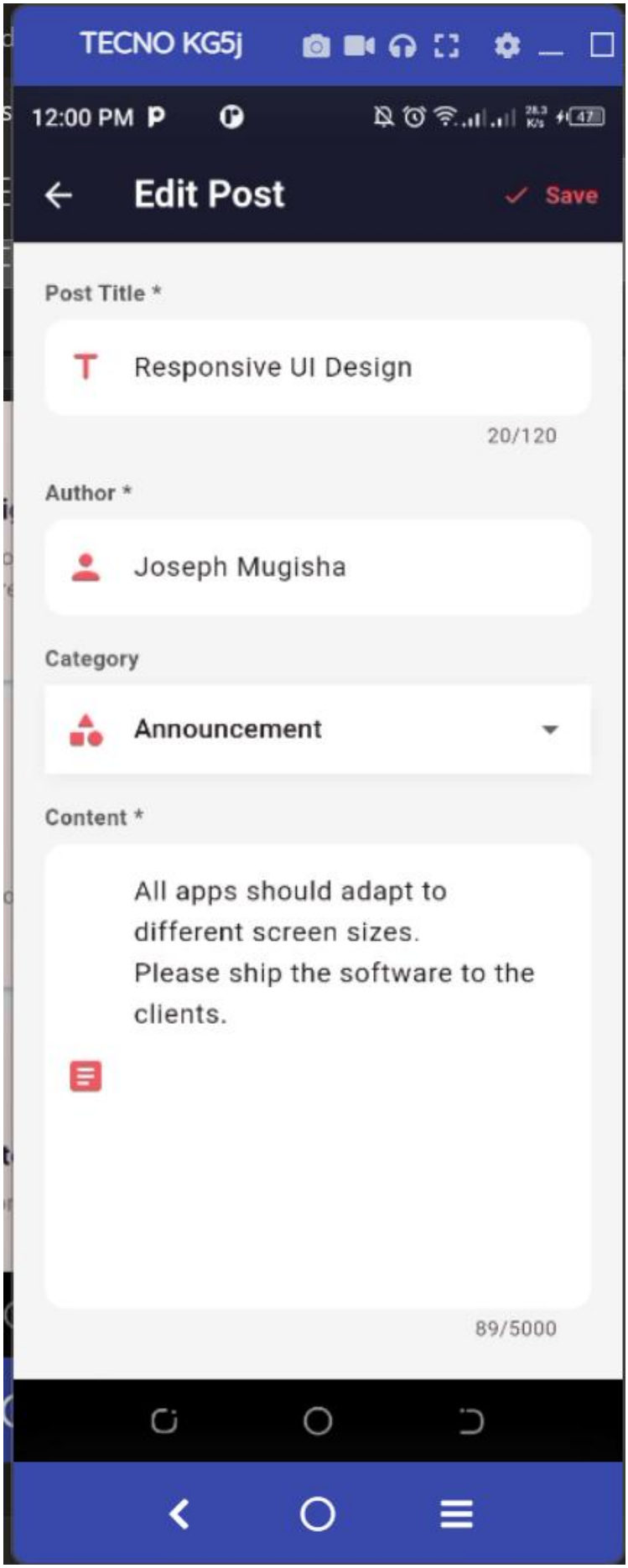

TECNO KG5j
12:00 PM
Offline Posts Manager
Search posts...
All
Announcement
Responsive UI Design
All apps should adapt to different screen sizes. Please ship the software to the clients.
Joseph Mugisha
Mar 18, 2026 · 12:33 PM
News
SQLite in Flutter
SQLite allows offline storage in Flutter apps.
Eric KAMANA
Mar 18, 2026 · 12:31 PM
Technology
Understanding State Man
Managing state is important fo
updates.
New Post

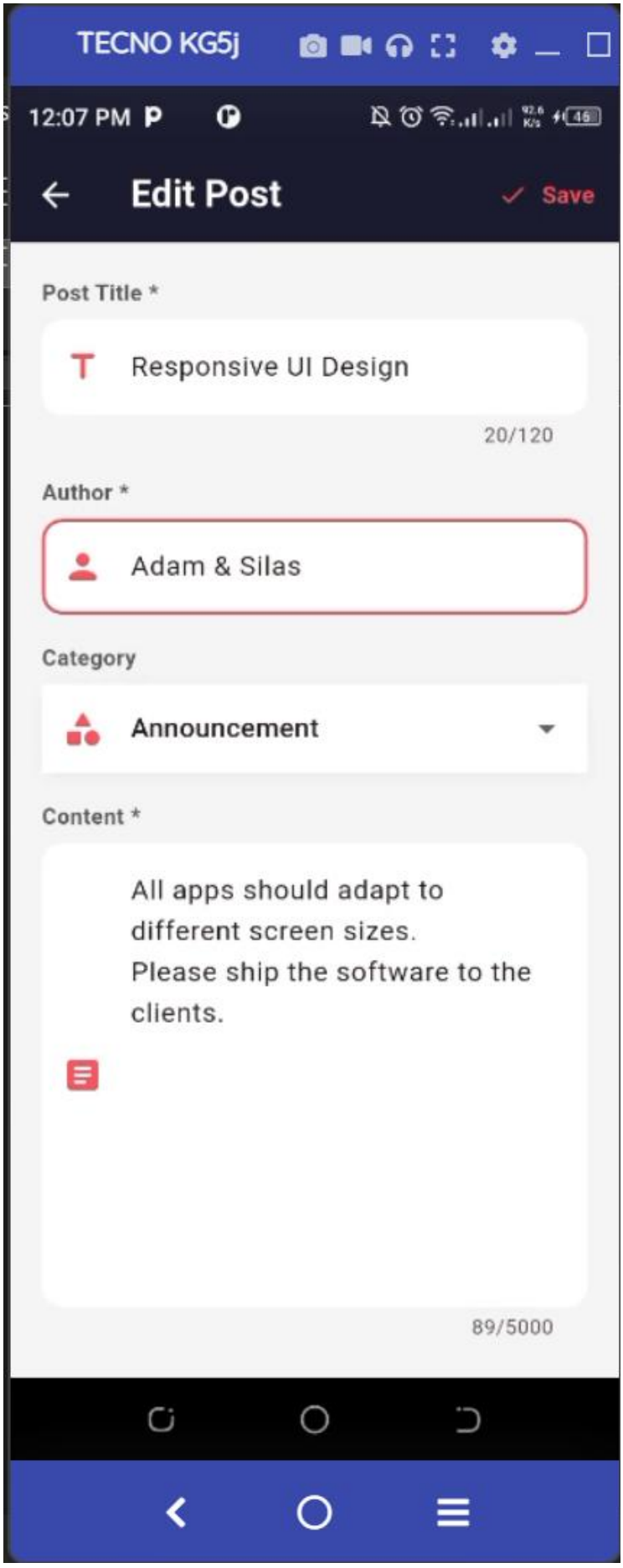

TECNO KG5j
12:02 PM
New Post
Save
Post Title *
Enter a compelling title...
0/120
Author *
Your name or pen name...
Category
General
Content *
Write your post content here...
0/5000

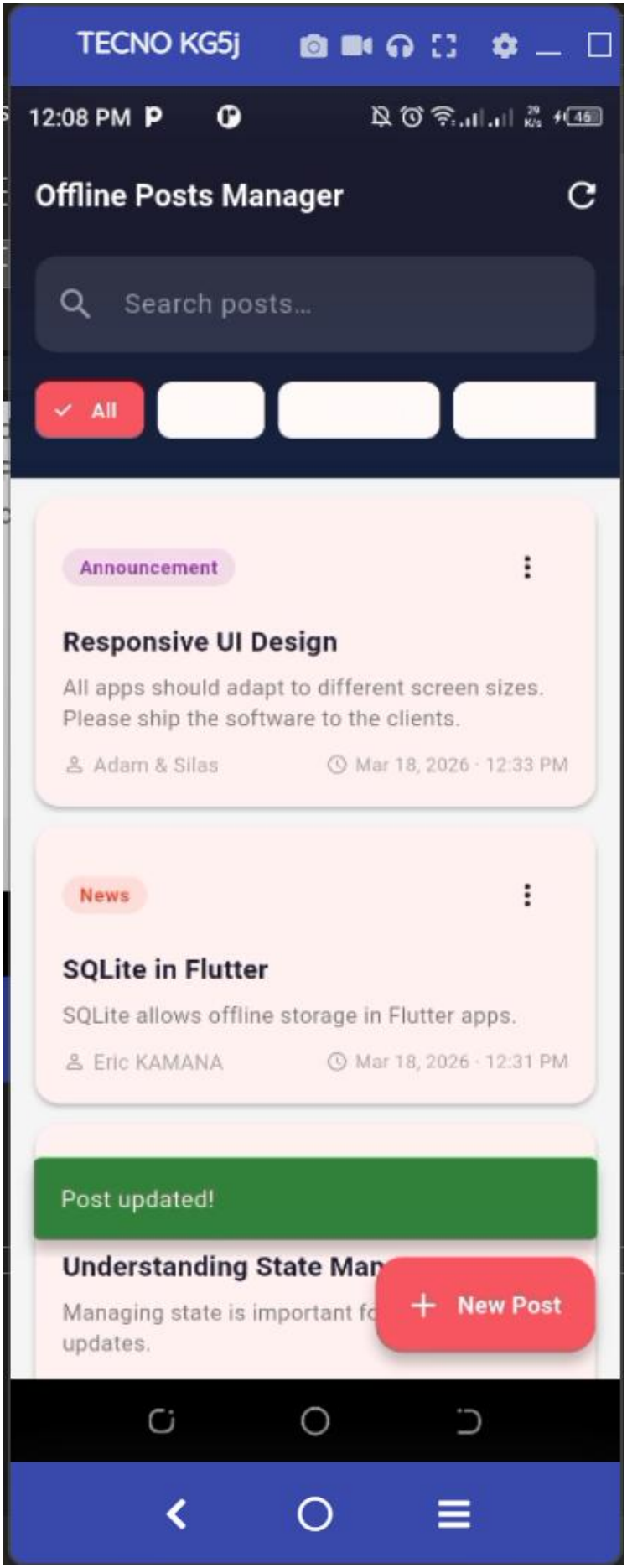

TECNO KG5j
12:10 PM
New Post
Save
Post Title *
Introduction of Flutter
23/120
Author *
Lect. Emmy
Category
News
Content *
Dear Year 3 CSE be prepared for
Flutter next week.
Have a wonderful day!
72/5000

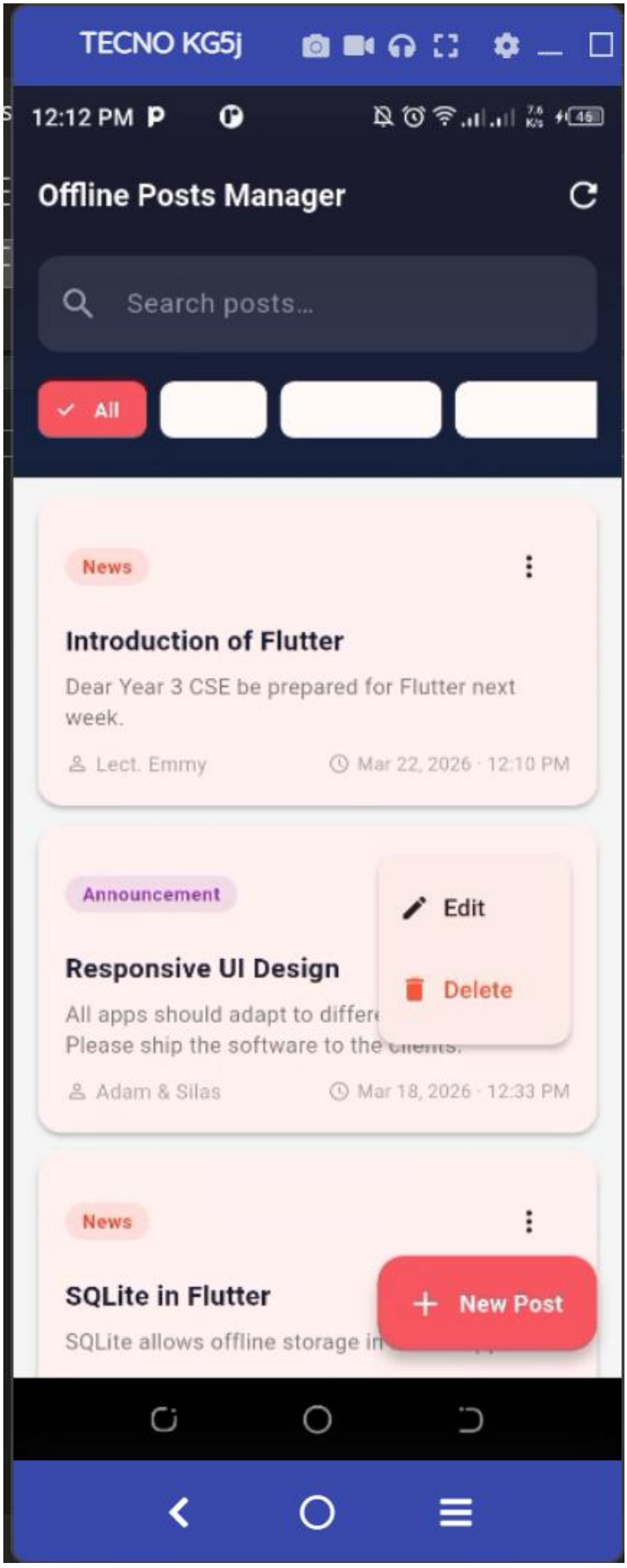

TECNO KG5j
12:10 PM
Offline Posts Manager
Search posts...
All
News
Introduction of Flutter
Dear Year 3 CSE be prepared for Flutter next week.
Lect. Emmy
Mar 22, 2026 · 12:10 PM
Announcement
Responsive UI Design
All apps should adapt to different screen sizes. Please ship the software to the clients.
Adam & Silas
Mar 18, 2026 · 12:33 PM
Post created!
SQLite in Flutter
SQLite allows offline storage in
New Post

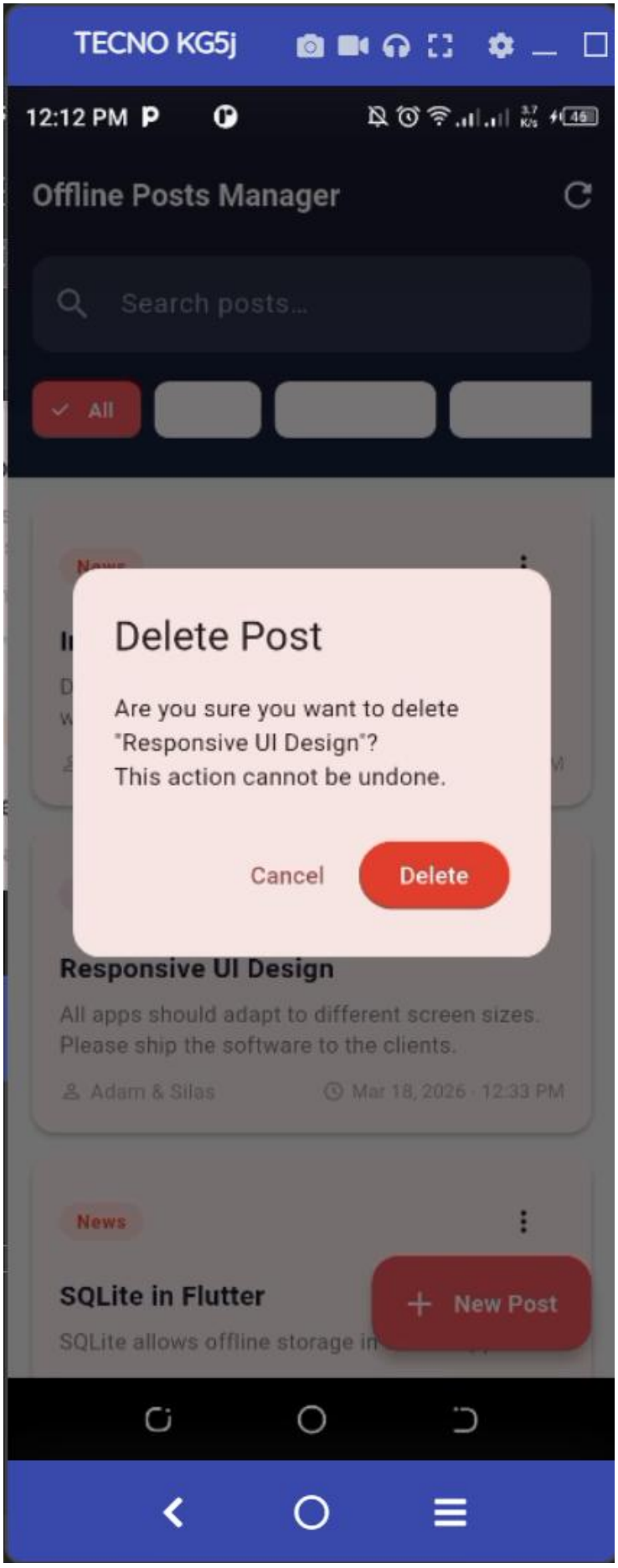

TECNO KG5j
12:03 PM
Offline Posts Manager
Search posts...
All
Announcement
Edit
Responsive UI Design
Delete
All apps should adapt to differe
Please ship the software to the clients.
Joseph Mugisha
Mar 18, 2026 · 12:33 PM
News
SQLite in Flutter
SQLite allows offline storage in Flutter apps.
Eric KAMANA
Mar 18, 2026 · 12:31 PM
Technology
Understanding State Man
Managing state is important fo
updates.
New Post

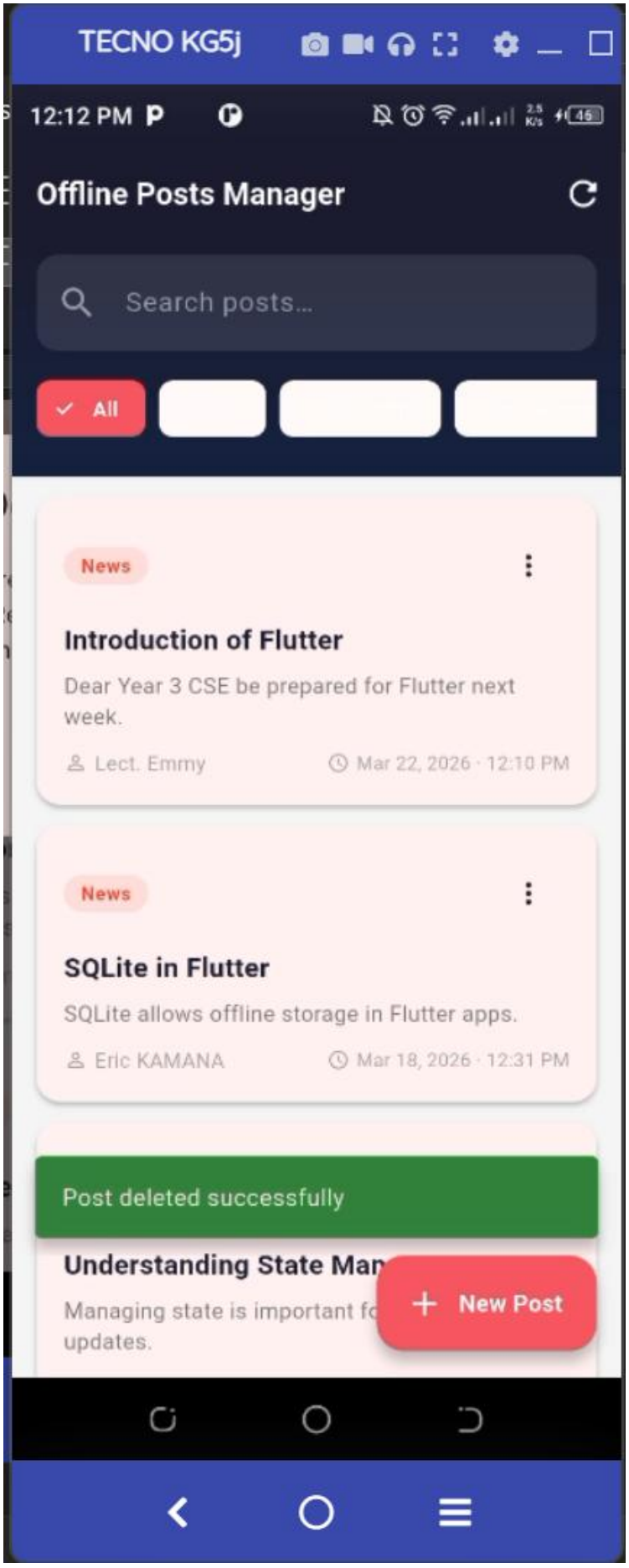

TECNO KG5j
12:00 PM
Edit Post
Save
Post Title *
Responsive UI Design
20/120
Author *
Joseph Mugisha
Category
Announcement
Content *
All apps should adapt to different screen sizes.
Please ship the software to the clients.
89/5000


TECNO KG5j
12:07 PM
Edit Post
Save
Post Title *
Responsive UI Design
20/120
Author *
Adam & Silas
Category
Announcement
Content *
All apps should adapt to different screen sizes.
Please ship the software to the clients.
89/5000


TECNO KG5j
12:08 PM
Offline Posts Manager
Search posts...
All
Announcement
Responsive UI Design
All apps should adapt to different screen sizes. Please ship the software to the clients.
Adam & Silas
Mar 18, 2026 · 12:33 PM
News
SQLite in Flutter
SQLite allows offline storage in Flutter apps.
Eric KAMANA
Mar 18, 2026 · 12:31 PM
Post updated!
Understanding State Man
Managing state is important fo
updates.
New Post


TECNO KG5j
12:12 PM
Offline Posts Manager
Search posts...
All
News
Introduction of Flutter
Dear Year 3 CSE be prepared for Flutter next week.
Lect. Emmy
Mar 22, 2026 · 12:10 PM
Announcement
Edit
Delete
Responsive UI Design
All apps should adapt to differ
Please ship the software to the
Adam & Silas
Mar 18, 2026 · 12:33 PM
News
SQLite in Flutter
New Post
SQLite allows offline storage in


TECNO KG5j
12:12 PM
Offline Posts Manager
Search posts...
All
Delete Post
Are you sure you want to delete "Responsive UI Design"?
This action cannot be undone.
Cancel
Delete
Responsive UI Design
All apps should adapt to different screen sizes. Please ship the software to the clients.
Adam & Silas
Mar 18, 2026 · 12:33 PM
News
SQLite in Flutter
SQLite allows offline storage in
New Post


TECNO KG5j
12:12 PM
Offline Posts Manager
Search posts...
All
News
Introduction of Flutter
Dear Year 3 CSE be prepared for Flutter next week.
Lect. Emmy
Mar 22, 2026 · 12:10 PM
News
SQLite in Flutter
SQLite allows offline storage in Flutter apps.
Eric KAMANA
Mar 18, 2026 · 12:31 PM
Post deleted successfully
Understanding State Man
Managing state is important fo
updates.
New Post